

Deep Learning

Midterm Project

Project Details

This Midterm Project is a practical application of the topics covered in the first part of the Deep Learning course. Students are expected to build, train, evaluate, and analyze deep learning models for an image-classification problem.

The project should demonstrate understanding of multilayer perceptrons (MLPs), activation functions, optimization, training dynamics, convolutional neural networks (CNNs), regularization, modern CNN architectures, and transfer learning.

Students may work:

- Individually,
- In pairs, **OR**
- In teams of up to three students.

Main Requirements

- **Problem Definition**

- Choose an image-classification problem.
- Clearly define the input, target classes, and objective of the project.
- Explain why the selected problem is suitable for deep learning.

- **Dataset**

- You may use a standard image dataset such as CIFAR-10, CIFAR-100, Fashion-MNIST, or another appropriate image dataset.
- Clearly describe:
 - * Dataset source;
 - * Number of samples and classes;
 - * Image dimensions and number of channels;
 - * Class distribution;
 - * Training, validation, and test split.
- Apply appropriate preprocessing such as normalization, resizing, and/or data augmentation.

- **Baseline Model: MLP**

- Implement an MLP baseline for the selected classification problem.
- Use at least two hidden layers.

- Clearly specify the activation functions, loss function, optimizer, learning rate, and number of epochs.
- Report training and validation loss/accuracy.
- **CNN Model**
 - Implement a CNN containing at least:
 - * Two convolutional layers;
 - * Activation functions;
 - * Pooling layers;
 - * A fully connected classification layer.
 - Explain the selected kernel sizes, number of filters, stride, and padding.
 - For at least one convolutional layer, calculate the output feature-map dimensions.
 - Report the total number of trainable parameters.
- **Optimization Experiment**
 - Train the same model using at least **three** optimizers chosen from: SGD, SGD with Momentum, RMSprop, and Adam.
 - Keep the model architecture and data split fixed for a fair comparison.
 - Compare:
 - * Training loss;
 - * Validation/test accuracy;
 - * Convergence behavior;
 - * Training time.
 - Plot the training curves for all optimizers.
- **Training Dynamics and Regularization**
 - Investigate at least **one** training problem or regularization technique:
 - * Vanishing gradients using Sigmoid versus ReLU/Tanh;
 - * Dropout;
 - * L2 regularization (weight decay);
 - * Learning-rate scheduling;
 - * Early stopping.
 - Explain how the selected technique affects training and generalization.
- **Modern CNN Architecture / Transfer Learning**
 - Use one modern pretrained CNN architecture such as **VGG16** or **ResNet18**.
 - Load pretrained ImageNet weights.
 - Replace the final classification layer so that it matches the number of classes in your dataset.
 - Fine-tune the model and compare its performance with your CNN.

- Briefly explain the idea of transfer learning and why pretrained features may help.
- **Model Evaluation**
 - Compare the MLP, your CNN, and the transfer-learning model.
 - Include at least:
 - * Test accuracy;
 - * Training and validation loss curves;
 - * Confusion matrix;
 - * Number of trainable parameters;
 - * Training time.
 - Discuss the trade-off between model complexity, training cost, and performance.
- **Feature Visualization**
 - Visualize at least eight learned filters from the first convolutional layer **or** activation maps for at least two sample images.
 - Briefly explain what information the early CNN layers appear to capture.
- **Report and Code**
 - Use Google Colab or Jupyter Notebook.
 - Combine code, explanations, tables, plots, and results in one reproducible notebook.
 - You may use either PyTorch or TensorFlow/Keras.
 - Code must be clearly organized and commented.

Report Structure

Your notebook/report should contain the following sections:

- **Title**
- **Abstract** (maximum 300 words)
- **Introduction (5%)**
 - Problem description;
 - Motivation;
 - Overview of the proposed approach.
- **Data and Preprocessing (10%)**
 - Dataset description;
 - Class distribution;
 - Train/validation/test split;

- Preprocessing and augmentation.
- **Methods (15%)**
 - MLP architecture;
 - CNN architecture;
 - Convolution settings and output-size calculation;
 - Modern CNN / transfer-learning architecture;
 - Loss function and training setup.
- **Optimization and Training Analysis (10%)**
 - Optimizer comparison;
 - Training curves;
 - Regularization or vanishing-gradient experiment.
- **Experiments and Results (15%)**
 - Quantitative comparison of models;
 - Confusion matrix;
 - Parameter counts and training time;
 - Feature/activation visualization;
 - Discussion of results.
- **Conclusion (3%)**
 - Summary of findings;
 - Limitations;
 - Possible improvements.
- **Writing / Formatting / Reproducibility (2%)**

Grading Scheme

Submission – 60%

- Introduction – 5%
- Data and Preprocessing – 10%
- Methods – 15%
- Optimization and Training Analysis – 10%
- Experiments and Results – 15%
- Conclusion – 3%

- Writing / Formatting / Reproducibility – 2%

Oral Defense – 40%

- Understanding of MLP and CNN architectures – 10%
- Understanding of convolution, stride, padding, pooling, and feature maps – 10%
- Understanding of optimization and training behavior – 10%
- Explanation of results and answers to questions – 10%

Submission Requirements

- Submit your work as a ZIP file.
- The notebook must run from beginning to end without missing code or files.
- All plots, tables, and final results must be visible in the submitted notebook.
- If an external dataset is used, include the code required to download/load it.
- File naming convention:

`FirstName.LastName.GroupNumber.ZIP`

Important Rules

- All experiments must be reproducible.
- The same train/validation/test split should be used when comparing models.
- Do not report results without showing the corresponding implementation and experiment.
- Students must be able to explain every important part of the submitted code during the oral defense.